Introduction

This document is a comprehensive documentation of Project UFO (**U**nscheduled **F**light **O**ut of service). The purpose of this document is to guide future development and integration of the project into other regions.

Who Should Read This Documentation?

This documentation is primarily intended for developers, both new and experienced, who wish to contribute to Project UFO or work on it for any reason. If you're looking to dive into the codebase, implement new features, fix bugs, or simply understand how the project works under the hood, you're in the right place. Even if you're not planning to contribute code, this documentation can still help you gain a deeper understanding of the project's architecture and design.

About Project UFO

A. Project Overview

Project UFO was started to improve communication, with respect to unscheduled maintenance, between the airline clients, Airbus, and the maintenance teams responsible for work on an aircraft. Additionally, it creates a standard prioritization for these maintenance issues.

B. Users

There are two main types of users for Project UFO.

1) ***Field Service Representatives* (FSR)**
   These individuals are the direct liaison between a particular client airline and Airbus. In situations where an airline has a question, concern or comment on a certain maintenance request, they will speak to their Field Service Representative. For the rest of the documentation, Field Service Representatives will be referred to as FSRs.
2) **End to End Users (E2E)**
   The liaisons between FSRs and the Design Office and SBC to a lesser extent. Help with identifying and responding to potential issues in the TR process
3) **SBC Super Users (Leadership, Dominik)**
   These users are part of the SBC team and are in charge of overseeing all the maintenance requests of a region. On a regular basis, they work to prioritize the current, open maintenance requests to determine where to devote the most resources and attention. For the rest of the documentation, SBC super users will be referred to as SBC users.

C. Detailed Project Description

Project UFO was started with the goal of achieving a few main outcomes:

1) Give the FSRs a better tool to manage and view the maintenance requests for their airline.
   a) Project UFO provides a more user friendly interface for FSRs to see all the current information on a specific maintenance request.
   b) Allows FSRs to easily jump between different maintenance requests.
   c) Allows FSRs to filter and search through all current maintenance requests.
2) Give the SBC users a standardized method for prioritizing and viewing open maintenance requests.
   a) Project UFO enables SBC users to dynamically configure a prioritization algorithm.
      i) On a week-by-week (or even day-by-day) basis, SBC users can determine how they want maintenance requests prioritized.
3) Create a more standardized channel for FSRs to communicate with SBC users when advocating/ justifying their airlines maintenance request

a) Project UFO creates several fields for a FSR to indicate to the SBC team certain properties of a maintenance request. This in turn, helps the SBC users more fairly prioritize across all the airlines in the region, while still allowing FSR input be the main driver of a maintenance prioritization.

4) SBC/Dominik view
   a) He

Architecture

A. Key Components

- **Datasets**
  - The datasets are the most foundational components of the project. They can be found in the 'Datasets' folder in the UFO file structure. The most pertinent datasets are in the directory 'ObjectBackingData'. The single most important dataset is 'UFOEntries'
- **Ontology Objects**
  - Project UFO is structured almost entirely around the Foundry Ontology. The rest of this documentation will be written with the assumption that the reader is familiar with the Ontology, which is documented and explained in detail here. The Ontology Objects are the central pieces of the Ontology. This project's main Object Types
    - UFOEntry
    - UFOicao
    - UFOfsr
    - Priority_Algorithm
    - MessageKPI
    - Escalations
    - [LOG] FSRInput

    These Ontology Object types can be seen and handled in the Ontology Manager application.
- **Ontology Actions**
  - Each Ontology Object Type has a set of Actions associated with it. Some Actions are associated with multiple Object Types. Every Ontology Object Type has a list of its associated Actions in its specific page in the Ontology Manager *(figure 1)*. Some of these Actions are defined with Ontology presets. Many of them, however, are custom Actions defined in one of the Code Repositories.
- **Code Repositories**
  - There are two crucial code repositories in the project file structure. They can both be found in the 'CodeRepositories' directory.
  - *DataTransformationRepository* is a code repository that builds the backing datasets for two of the Object Types: UFOEntry and UFOicao. Once in the repository, the respective scripts can be found in the directory at "transforms-python/src/myproject/datasets". These scripts are written in Python and utilize PySpark to build and interpret datasets.
  - *UFO_OntologyObject_Functions* is a repository that is responsible for creating the backing logic for the more specific Action types for the Ontology Object Types. Once in the repository, the script defining this backing logic is found at functions-typescript/src/index.ts. This script is written in TypeScript.
- **Workshop Modules**
  - Workshop is a Skywise UX application. The rest of this documentation will be written with the assumption that the reader is familiar with Workshop, which is

documented in more detail here. There are currently two Workshop Module types that are a part of Project UFO. They can be found in the directory 'WorkshopModules' in the UFO file structure.

- *PrioritizationConfiguration* is the Workshop Module that allows SBC users to dynamically create, edit and test new prioritization algorithms.
- *UFODashboard* is the Workshop Module where FSRs will interact with maintenanceRequests in order to communicate with their airlines and give their input to SBC prioritization.

**Note that the names of Workshop Modules might not be named exactly what is written in the Documentation, especially as more regions begin to adopt the project. Naming convention will likely be something along the lines of *RegionName_PrioritizationConfiguration* or *RegionName_UFODashboard.* It will, however, be very intuitive to make distinctions between the two Workshop Modules.

- **Automations**
  - Automations are scheduled executions of Ontology Actions. They can be found in the Automations Directory in the UFO file structure.

B. Component Interactions

- **Datasets and Ontology Objects**

  Datasets act as the initial backing data for Ontology Object Types. They define the properties and data types in a specific Object Type. They do not interact much with the Ontology beyond initializing the objects and populating some data.
  - Ontology Object Types, when created, are backed by a specific dataset. The UFOEntry Object Type, for example, is backed by the UFOEntries dataset. The columns within the dataset define the properties of each Object Type and each row in the dataset defines a new instance of the Object Type.
    - For example, the *dataset* UFOEntries has 53 columns, one of which is Operator_ICAO. The *Ontology Object Type,* UFOEntry, has a property for Operator_ICAO. The *dataset* has approximately 40,000 rows. The *Ontology Object Type* has 40,000 instances of UFOEntries,each with an Operator_ICAO property, defined by the value in the dataset.
  - Some of the Object Types have writebacks enabled, which allows edits to be made to a specific instance of an Object Type. These writebacks used to be stored in separate datasets when the Objects were created with Object Storage v1. Now that all the Object Types are migrated to Object Storage v2, the writebacks are stored within the Ontology backend.
  - The writebacks stored in the Ontology backend will supersede whatever is present in the dataset.
    - For example, the UFOEntries *dataset* has a column for Priority so that the *Object Type* has a property for Priority. By default, the *dataset* populates every row in the Priority column with "Green". However, edits made to a particular instance's Priority property will be retained even if the *dataset* is populated with a "Green" value.
  - The UFOEntry Object Type is the Object Type that is most affected by the backing dataset. The UFOfsr and PriorityAlgorithm Object Types exist almost entirely in the Ontology backend. Datasets still exist to back these Object Types, but they are only present to define the properties.

        - For example, the PriorityAlgorithm backing *dataset* has only one row. It is populated with dummy data. This *dataset* was created to define the properties and data types of properties for the PriorityAlgorithm *Object Type*. The rest of the *Object Type* instances of PriorityAlgorithm are defined in a separate interaction and are stored in the Ontology Backend.
    - The UFOicao Object Type is partially defined by its backing dataset. The Object Type instances are defined by the rows in the dataset, but the rest of the pertinent data is populated by Object Type edit functions.
    - All datasets that act as backing datasets for an Ontology Object Type can be found in the Datasets/ObjectBackingData directory from the UFO file structure

- **Datasets and Code Repositories**
  Some of the Datasets act as inputs to the transformation logic in the DataTransformationRepository. Other datasets are outputs of the transformation logic.
    - The datasets only interact with the DataTransformationRepository. There is no interaction between the datasets and the UFO_OntologyObject_Functions.
    - Some datasets in the project structure act as inputs to the DataTransformationRepository.
        - SmallSample is a dataset found in the NonOntologyDatasets directory in the UFO file structure. This is a dataset that is produced by a Contour Analysis on the TechRequest_dossier_index_metadata dataset. SmallSample acts as an input to the python transform UFOEntries.
        - UFOEntries is a dataset that acts as an input to the python script ICAOObjects **
    - The DataTransformationRepository outputs two datasets
        - UFOEntries.py outputs the UFOEntries dataset, which acts as backing dataset to the UFOEntry Object Type (see above) **
        - ICAOObjects.py outputs the icaoUFOobjects dataset, which acts as the backing dataset for the UFOicao Object Type (see above)

    ** Note that the UFOEntries dataset is both an output of one transformation and an input to another

- **Ontology Objects and Ontology Actions**
    - UFOEntry Object Type
        - A *UFOEntry instance* can have the following properties edited through the *FSRInputDriver* Action Type: Priority, Aircraft Status, Focal, New Request Date. The *FSRInputDriver* Action Driver then recalculates the Priority Score property and recalculates properties in *UFOIcao* Object Type instances. The *FSRInputDriver* Action Type is triggered through the UFODashboard Workshop Module.
        - A *UFOEntry instance* can have its Comments Property updated through the *CommentUpdate* Action Type. This action is triggered through the UFODashboard Workshop Module.
        - A *UFOEntry instance* can have its dossierStatus property updated through the *ToggleStatus* Action Type. This action is triggered through the UFODashboard Workshop Module
        - A set of *UFOEntry instances* can have their dossierStatus property updated through the *UpdateClosed* Action Type. This is done through Foundry Automate.
        - A set of *UFOEntry instances* can have their Priority Score calculated with the active configuration and have their Priority Score property updated

with the calculated score through the *ConfigureAllScores* Action Type. This is done through Foundry Automate.

- A single *UFOEntry instance* can have its Priority Score calculated with the active configuration and have its Priority Score property updated with the calculated score through the *GetGlobalScore* Action Type. There are no explicit triggers for this Action Type but the backing logic is called by the *FSRInputDriver* Action Type in the UFODashboard Workshop Module.
- A set of UFOEntry instances can have their Priority Score calculated with any of the existing configurations (not just the active configuration) and have their Priority Score property updated with the calculated score through the *PriorityTester* Action Type. This function is triggered in the PrioritizationConfiguration Workshop Module where it is applied to a Scenario context. More information on Scenarios can be found here. Essentially, in the Scenario context, the Action Type will not make changes to the actual Object instances, but make changes to the Object instances within the scenario. With respect to the *PriorityTester* Action, this allows SBC users to experiment with different *Prioritization_Algorithm Object Type instances.*
- A *UFOEntry instance* can be deleted through the *DeleteUFO* Action Type. There are no triggers for this action type. It must be done through the Ontology Manager.
- A set of *UFOEntry instances* can all be deleted through the *DeleteMultiple* Action Type. There are currently no triggers for this action type. It must be done through the Ontology Manager.
- A set of *UFOEntry instances* can have their isFavorite property changed by the *FSR_Favorites_Display* Action Type. This Action Type is triggered by the initial load up of the UFODashboard Workshop Module. The *FSR_Favorites_Display* uses a *UFOfsr instance* as an input. Like the *PriorityTester* Action Type, the *FSR_Favorites_Display* Action Type is applied to a Scenario Context. The isFavorite property is only adjusted within a particular instance of a Workshop Module.

- UFOicao Object Type
  - Although not explicitly declared in the UFOicao Object Type's Action Types, a set of *UFOicao instances* can have the following properties edited through the *ICAOCount* Action Type: Amber Cap Met, Amber Count, Ambers Allotted, Closed Count, Open Count, Red Cap Met, Red Count. The *ICAOCount* Action Type calculates these properties based on aggregated properties of an input set of *UFOEntry instances*.
- UFOfsr Object Type
  - A new *UFOfsr instance* can be created with the *NewFSR* Action Type. There are currently no triggers for this action type. It is manually executed through the Ontology Manager when a new UFOfsr instance is necessary
  - A *UFOfsr instance* can be deleted through the *DeleteFSR* Action Type. There are no triggers for this Action Type.
  - A *UFOfsr instance* can have its ICAOs property set with the *SetICAOS* Action Type. There are no triggers for this Action Type. The ICAOs property is a string array and the list of potential values in the string array is populated with the Operator ICAO property from the *UFOEntry* Object Type
  - A *UFOfsr instance* can have its Favorites property edited with the *addFavs* and *removeandUpdateFSRFavs* Action Types.The Favorites

property, a string array, is appended to with the *addFavs* Action Type. The *removeandUpdateFSRFavs* removes values from the string value in the Favorites property. The Favorites property is a string array and the list of potential values in the string array is populated with the DossierId property from the *UFOEntry* Object Type

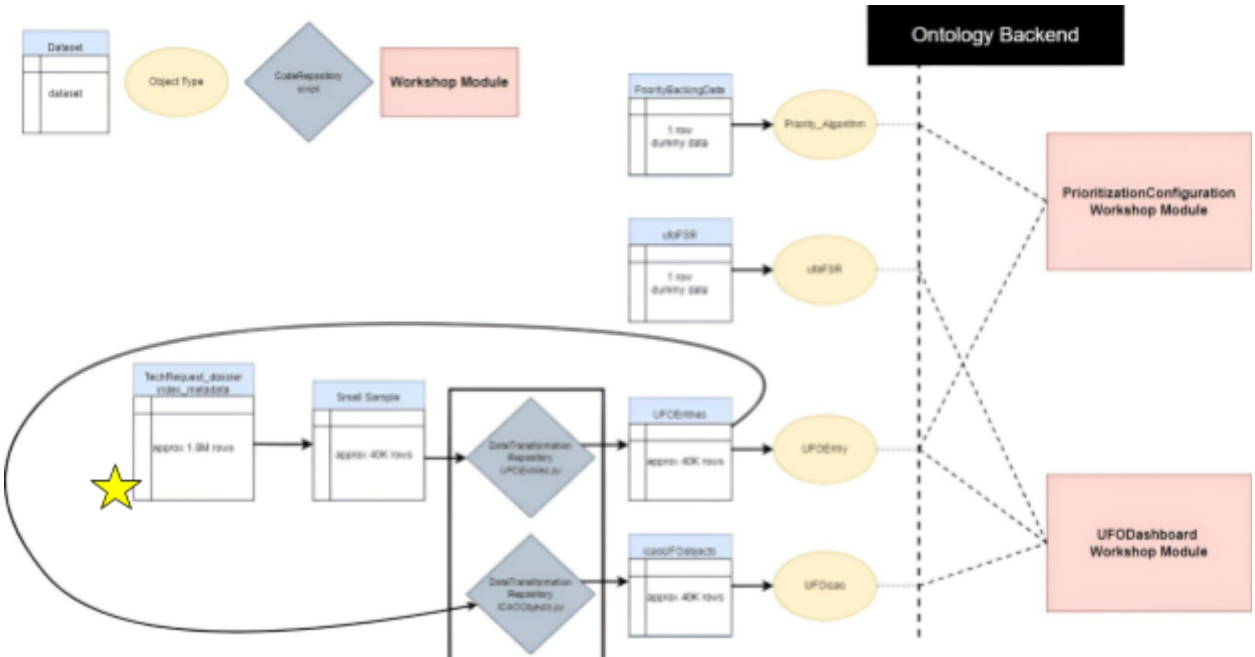
- Priority_Algorithm Object Type
    - A *Priority_Algorithm instance* can have any of its Priority Parameter properties (1-10) changed or initialized with the *ChangeorInitiatePriorityParameter(1-10)* Action Type. This is triggered through the PrioritizationConfiguration Workshop Module.
    - A *Priority_Algorithm instance* can have any of its Priority Parameter (1-10) properties configured with the *ConfigurePriorityParameter (1-10)* Action Type. This involves editing the Parameter(1-10) L(1-5) Property. This is triggered through the PrioritizationConfiguration Workshop Module.
    - A *Priority_Algorithm instance* can be created with the *CreatePriorityAlgorithm* ActionType. This action is triggered through the PrioritizationConfiguration Workshop Module.
    - A *Priority_Algorithm instance* can be deleted through the *DeletePriorityAlgorithm* Action Type. This action is triggered through the PrioritizationConfiguration Workshop Module.
    - A *Priority_Algorithm instance* can have its MainConfigFlag property set to 1, indicating that this particular instance is the active configuration. This is done with the *setMainConfiguration* Action Type. The *setMainConfiguration* ActionType also pulls every other *Priority_Algorithm instance* and sets their mainConfigFlag property to 0 so that there is only one *Priority_Algorithm instance* that is active at one time.

- **Ontology Objects and other Ontology Objects**
    - UFOEntry and UFOicao
        - *UFOEntry instances* can have their Priority property changed. The Amber Cap Met and Red Cap Met properties in the corresponding *UFOicao instances* constrain the potential values for this property change. This interaction occurs in the UFODashboard Workshop Module.
    - UFOEntry and UFOfsr
        - In the UFODashboard Workshop Module, a particular *UFOfsr instance* filters the favorite *UFOEntry instances*. This interaction occurs in the UFODashboard Workshop Module.
    - UFOEntry and Priority_Algorithm
        - *UFOEntry instances* will have their Priority Score calculated by an Action Type. The Action Type determines which *Priority_Algorithm instance* is active and applies the specific configuration to calculate the Priority Score.
- **Ontology Objects and Workshop Modules**
    - UFODashboard Workshop Module
        - The primary purpose of the *UFODashboard Workshop Module* is to display a set of *UFOEntry Object Type instances*. The *UFODashboard* selects a certain subset of all existing *UFOEntry instances*. This subset is statically defined for each unique Workshop Module.
            - For example, the North American *UFODashboard Workshop Modul*e only displays *UFO instances* with North American ICAO codes in their Operator ICAO property.

        - The UFODashboard Workshop Module also identifies the current user and finds the corresponding UFOfsr instance based on User ID. It uses the UFOfsr instance to isolate certain UFOEntry instances in the favorites tab.
    - PrioritizationConfiguration Workshop Module
        - The *PrioritizationConfiguration Workshop Module* exists so that SBC users can create, edit and delete *PriorityAlgorithm instances*.
- **Ontology Actions and Code Repositories**
    - Some of the more in depth Action Types have their backing logic defined in the *UFO_OntologyObject_Functions Code Repository*. Each Action Type, through the Ontology Manager, has a Rules section where it shows which defined function is backing the Action Type (if there is one).
        - For example, the FSRInputDriver Action Type is defined by the fsrEntryDriver() function written in the *UFO_OntologyObject_Functions Code Repository.* *(figure 2)*
- **Ontology Actions and Workshop Modules**
    - As mentioned in the section covering interactions between Object Types and Action Types, many of the Actions are executed in Workshop Modules.
    - All Action Types can be executed in the Ontology Manager application. Some of the action types, however, require a set of input, most obviously where the backing function requires a set of parameters, and the Workshop Module will automatically populate those inputs when possible.
        - For example, some of the functions act on whole sets of UFOEntry instances, aggregating values across the set. In the context of a regional UFODashboard, the UFOEntry instance set only needs to reflect the region at hand. Actions executed within this dashboard will automatically handle this filtering.
- **Automations and Action Types**
    - There are 3 Action Types that run every time new data is loaded into Skywise. This schedule is implemented using Automations.
    - There are Automations on *ConfigureAllScores, ICAOCount* and *UpdateClosed*

C. Data Flow

Below is a diagram showing the general flow of data, from the raw input data, TechRequest_dossier_index_metadata (indicated with the yellow star), to the Workshop Modules. It is worth noting that the TechRequest_dossier_index dataset is outside the scope of this project. It is the Skywise Wide dataset pulling metadata from TechRequests.



UFODashboard

A. Dashboard Components

- Dashboard Components can be found here

0. ~~High Level View of all UFOEntries~~
   a. ~~This section uses an Object List Widget to show every UFOEntry instance that was loaded into the Workshop Module. The High Level view shows some properties for each entry.~~
   b. ~~Many of these columns are prone to hold many Null values. With some exceptions, non null values are highlighted in a light blue and null values are shown very faintly. Columns that have further conditional formatting can be seen through the properties page in the Ontology Manager application (*figure 3)*~~

~~1. UFOEntry Properties~~

~~When an UFOEntry is selected from the High Level view of all UFOEntries, it is shown in the Object View Widget in this section. The Object View Widget shows almost every property of the active UFOEntry. The exact configuration for the Object View Widget for the UFOEntry Object Type can be edited and viewed through the Ontology Manager Application *(figure 4).*~~

~~2. Favorite UFOentries List~~

~~An FSR can add or remove specific UFOEntry instances to their personal list of favorites. The list of UFOEntry instances is displayed in this panel. Each UFOEntry in this list has a checkbox associated with it.~~

~~3. Add or Remove Favorites~~

~~This text entry and button group allows the FSR to add or remove UFOEntry instances from their Favorites. When a UFOEntry in the Favorite UFOEntries List has its checkbox marked, it is automatically populated in the text entry.~~

~~4. General Filter List~~

~~This component holds a Filtering widget that allows the user to filter the Object List Widget in Section 1. All properties have filtering capabilities, and users can add or remove filters. Filters are stackable. The Workshop Module is automatically loaded with filters.~~

5. ~~High Level View Tabs~~

   ~~Here the user can switch between the High Level View of all UFOEntry instances and the UFOEntry instances in the FSR's favorite list~~

6. ~~FSR Input Button~~

   ~~The user selects this button to execute the *FSRInputDriver* Action Type and make edits to the UFOinstance that is selected in the High Level View~~

7. ~~Comment Panel~~

   ~~This section is where the user makes comments that are specifically tied to the UFOEntry selected in the High Level View. When a comment is made, the comment is also appended to the string Array in the UFOEntry's Comments properties.~~

8. ~~Mark as Closed~~

   ~~This button allows the user to edit the Dossier Status property of the UFOEntry instance that was selected in the High Level View. When the button is clicked, the ToggleStatus Action Type is triggered. If the UFOEntry instance is 'OPEN', this action will change the Dossier Status property to 'CLSD-u' to indicate that it was closed by a user. If the UFOEntry instance is CLSD-u, this action will change the Dossier Status property to 'OPEN'. If the UFOEntry instance is 'CLSD', this action will not be available.~~

9. ~~TechRequest Link~~

   ~~This button will link the user to the TechRequest Page that corresponds to the UFOEntry that is selected in the High Level View.~~

B. Variables

The variables within the dashboard are defined here.

ObjectTypes in Depth

A. UFOEntry

The UFOEntry Object Type is where the bulk of the dashboard functionality is centered, and the most important part of Project UFO as a whole. For most intents and purposes, each UFOEntry can be understood as a TechRequest Dossier. There is a one to one link between each UFOEntry and a unique TechRequestDossier.

Each UFOEntry has 53 properties. 42 of these properties have a direct relationship with the columns in the TechRequest_dossier_index_metadata *(see datasets)*, and the property names in the Object Type are intuitively related to the column names in the Dataset. The 12 remaining properties are added during the dataset build (see Code Repositories) and are detailed below:

1. UniversalKey
   a. This is an integer representation of the DossierId. In other words, the Dossier ID is stored twice in the UFOEntry Properties. Once as a string in 'Dossier_ID' and once as an integer in 'UniversalKey'
2. Priority
   a. This property allows the FSR to classify the UFOEntry into one of 3 priority categories: Green, Amber, Red, where Red is the highest priority and Green is the lowest priority. This property is Green by default. This property is edited through the FSRInputDriver Action Type.

3. Aircraft Status
    a. This property allows the FSR to classify the UFOEntry into one of 3 AircraftStatus categories: AOG, Heavy Maintenance Visit, Heavy Maintenance - WSP (work stoppage). This property is empty by default. This property is edited through the FSRInputDriver Action Type.
4. Focal
    a. This property allows the FSR to add the name of an individual or entity in charge of a particular UFOEntry. If a UFOEntry requires special attention or there is a particular individual in charge of its processing, it is helpful to have the responsible party logged in the UFOEntry. This property is empty by default. This property is edited through the FSRInputDriver Action Type.
5. Days Since Newest Dossier Action
    a. This property holds a value that represents the number of days that have passed since the most recent Dossier Action, whether it was a Dossier Closing, Creation, Submission or Update. This is used in determining if a UFOEntry should be marked as Closed by the Mark as Closed Action Type.
6. New Request Date
    a. This property allows the FSR to add a new value for Message Soonest Request Date without actually changing the Message Soonest Request Date property. Sometimes, the client airline needs an aircraft back in service by a certain date but doesn't include enough buffer time between this return to service and the requested message reply date. For instance if an airline needs an aircraft fully ready to fly on January 10th, and they request a message response on the 10th, they are not including enough time for approval documents to be processed for the plane to be flight ready. This potential issue could be mitigated by the FSR if the New Request Date is pushed back to January 8th. By default, this value is populated with the same value in the Message Soonest Request Date property. Any prioritization algorithm that acts on Message Soonest Request Date is actually reading the New Request Date value.
7. GlobalPriorityScore
    a. This property holds an integer value between 0 and 100. This property is calculated by the backing priority algorithm. The higher the score, the higher the priority for the particular UFOEntry. The High Level View, holding the list of UFOEntries is sorted by priority score so that the UFOEntries at the top of the list are the ones with the highest priority.
8. Comments
    a. This property holds all the comments made in the comment tab in the Field Service Input through the UFODashboard. This is used in the case that the high priority UFOEntries are exported to a csv file and they want the comments retained.
9. RTS
    a. This property holds the Return To Service Date. The Return To Service Date is the day that the aircraft needs to be ready to fly. This value is extracted from the Label property in the DataTransformation Repository. When the Label holds the char sequence ‘RTS’, the subsequent chars are scraped and used to build a Date Object.
10. Is Favorite

        a. This property is used to build the favorite list in the UFODashboard. When a FSRobject has a DossierID in their favorites property, the Is Favorite boolean is set to True by the FSR_Favorites_Display Action Type, and the Dashboard filters the UFOEntries on this property. It is important to note, however, that this action always occurs within a Scenario. In other words, the Is Favorite property is never actually changed, the Dashboard simply acts as if it was set to True. This allows multiple FSRs to use and have separate sets of Favorite Dossiers.
    11. Closed by Back Office
        a. This is a boolean value that is set to True if the Dossier has been closed in the backing Dataset. The nature of Ontology Edits is such that they will always override backing dataset. In the context of the UFODashboard, when a UFOEntry is marked as closed, its Dossier Status will change and will override the value that the dataset holds. If, however, this UFOEntry is presented to the back office and the corresponding Tech Request is actually closed, the UFOEntry will continue to hold the value “CLSD-u”, indicating that it is not closed in the backing Tech Request. The Closed by Back Office property signifies whether or not the UFOEntry should be marked as closed. The UpdateClosed Action Type finds any UFOEntries that have this inconsistency and mark the UFOEntries as “CLSD”
    12. SBC Bump
        a. This is an integer value that is added to the GlobalPriority Score. It can only be edited by the SBC team and is used to break any Priority ties.

B. UFOfsr

The UFOfsr Object Type represents the users of the Dashboards. With respect to the types of users, this Object Type is slightly misnamed as both FSRs and SBC users have a corresponding UFOfsr instance. This distinction is made in the Object properties which are defined below.

1. Name
    a. This is the name of the user. It is used in the UFO Dashboard in the Favorites list (component 2).
2. User ID
    a. This is the Skywise ID of the user. This property is how the Dashboard identifies the user and isolates the corresponding FSR Object Type
3. Favorites
    a. This is a string array holding the Dossier IDs of the favorite UFOEntry instances for the specific UFOfsr Object Type. This list is used in the *FSR_Favorites_Display* Action Type in a scenario context to populate the My Favorites tab with the favorite UFOEntries for a UFOfsr.
4. ICAOS
    a. This is a string array holding ICAO codes. When a UFOfsr Object Type is using the UFO Dashboard, the UFOEntries are automatically filtered along these ICAO codes.
5. Role
    a. This is a string that differentiates between FSRs and SBC users.

~~**C. UFOicao**~~

~~The UFOicao Object Type represents a particular airline for the purpose of the dashboard. The primary use of the UFOicao Object Type is to keep track of how many UFOEntries have had their Priority property escalated by an FSR. Based on this number, the UFODashboard may restrict the options for Priority Escalation. This is in place to~~

~~prevent one airline within the dashboard from gaining an unfair advantage in prioritization by marking all their UFOEntries as Red priority.~~

1. ~~ICAO~~
    a. ~~This is a string value that holds the name of the ICAO the Object instance represents, eg: “DAL” for Delta Air Lines.~~
2. ~~Open Count~~
    a. ~~This is an integer that holds the number of UFOEntry instances that correspond to the following filter~~
        i. ~~(Operator ICAO == ICAO) && (DossierStatus == “OPEN”)~~
3. ~~Closed Count~~
    a. ~~This is an integer that holds the number of UFOEntry instances that correspond to the following filter~~
        i. ~~(Operator ICAO == ICAO) && (DossierStatus == “CLSD”)~~
4. ~~Ambers Count~~
    a. ~~This is an integer that holds the number of UFOEntry instances that correspond to the following filter~~
        i. ~~(Operator ICAO == ICAO) && (DossierStatus == “OPEN”) && (Priority == “Amber”)~~
5. ~~Amber Allotted~~
    a. ~~This is a dynamically calculated integer. It is equal to 10% of the value in the Ambers Count property.~~
6. ~~Amber Cap Met~~
    a. ~~This is a boolean that is equivalent to the following:~~
        i. ~~Ambers Count > Amber Allotted~~
7. ~~Red Count~~
    a. ~~This is an integer that holds the number of UFOEntry instances that correspond to the following filter~~
        i. ~~(Operator ICAO == ICAO) && (DossierStatus == “OPEN”) && (Priority == “Red”)~~
8. ~~Red Cap Met~~
    a. ~~This is a boolean that is equivalent to the following:~~
        i. ~~Red Count > Red Allotted~~
            1. ~~Note that there is no property for Red Allotted. This is a static value, where every Airline has up to 3 Red Priority UFOEntries. This value is found in the logic backing the UpdateICAO Action Type.~~

~~It is worth noting that, in the backing logic that defines the boolean Cap properties, the values that represent the upper limit (the Ambers Allotted or the static Red Allotted) are represented by an integer value 1 less than the actual value. For instance, an ICAO with 230 open UFOEntry instances will have an Amber Allotted value of 23, but the backing logic will hold a value of 22. This allows the cap to operate on a strict “greater than” (>) rather than a “greater than or equal” to (>=), which can get buggy with respect to 0.~~

~~The UFOicao interacts with the Workshop Module. Its Amber Cap Met and Red Cap Met properties populate parameters in the fsrInputDriver Action Type. These parameters don’t have any use in the backing logic. They do, however, play a part in the Action Form, where the user defines certain values for the UFOEntry, by defining certain options the user can choose from when assigning Priority.~~

D. Priority_Algorithm

The Priority_Algorithm Object Type defines how the Prioritization Logic should act on the set of UFOEntries. It defines which UFOEntry properties should be evaluated and along which parameters those properties should be scored.

1. Algorithm Name
    a. This is the name of the Priority_Algorithm instance. It is how each instance is identified. Once set, it cannot be changed.
2. MainConfigFlag
    a. This is an integer value that holds either 1 or 0. A value of 1 signifies that the Priority_Algorithm instance is the active configuration. When the prioritization logic occurs, it uses the properties and parameters defined in the active Algorithm. This value is set with the *setMainConfiguration* Action Type, which ensures that only one Priority_Algorithm instance will have a value of 1 in this property.
3. Priority Parameter *X*
    a. There are 10 properties that follow this naming convention, differentiated by the integer appended to the end. Currently this is demonstrated with *X,* but in fact, there is a property for Priority Parameter 1, Priority Parameter 2 …. Priority Parameter 10.
    b. This property is a string that represents a property in the UFOEntry Object Type that should be evaluated in the prioritization logic.
    c. This property can be empty. Every Priority_Algorithm instance can define up to 10 properties, but it is not necessary to do this.
4. Parameter*X* L1-L5
    a. Each Priority Parameter property, as defined immediately above, has 5 levels associated with it. These levels are represented by this property. This naming convention represents 50 different properties:
        i. Parameter1 L1 – Parameter10 L5
    b. This property represents how a value within the defined UFOEntry property should be scored.
        i. For example: If Parameter1 is defined as “Operator ICAO”, Parameter1 L1 could be defined as “DAL” so that the backing prioritization logic assigns a certain score to UFOEntry instances with DAL in the Operator ICAO.
5. Span Bool *X*
    a. There are 10 properties that follow this naming convention, differentiated by the integer appended to the end. Currently this is demonstrated with *X,* but in fact, there is a property for Span Bool 1, Span Bool 2 …. Span Bool 10.
    b. This property is a Boolean value that dictates whether or not Priority Parameter *X* is a span property. By default, this property is False.

The idea behind creating a whole new Object Type for Prioritization_Algorithm, rather than hard code the properties and parameters in the backing logic, was to allow SBC users to create several, unique Prioritization_Algorithm instances, giving them the flexibility to choose a certain prioritization configuration on a week by week basis.

More information on how each Prioritization_Algorithm is used in the backing logic can be found in the Prioritization and Dashboard section of this documentation.

Prioritization and Dashboard

A. High Level Prioritization sequence

Typically the prioritization process occurs on a set of UFOEntry instances and it will perform as much batch processing as it can before approaching each instance in the set individually. The prioritization begins by finding the active Priority_Algorithm and creating the configuration based on the properties and parameters defined in the Priority_Algorithm. The properties are sorted into different types, which determines which backing logic should be applied to it. For instance, a property that holds Date values will be evaluated differently than a property that holds String values. The backing logic also ensures that, for up to 10 properties, the total sum of the weights is 100% and the weights are decreasing. Ex: if there are only two properties being scored, the weights are 66.6% for Property 1 and 33.3% for Property 2. If there are 5 properties, the weights are 33.3%, 26.7%, 20%, 13.3% and 6.7%, respectively.

For each UFOEntry in the set, it finds the values stored in the defined properties. For each value, the backing logic assigns a score based on the levels defined in the Prioity_Algorithm. This score is multiplied by the weight of the parameter. This multiplied value is added to the UFOEntry's overall Global Score.

B. Prioritization Logic

The actual logic behind the process of assigning priority scores can be found in the *UFO_OntologyObject_Functions* Code Repository. Within this repository, there is extensive commenting on each step of the process. The process will be explained here, but any further confusion can probably be solved by looking at the pertinent code and the comments.There are a number of helper functions involved in the process of prioritizing. The main driver, responsible for coordinating the execution of these helper functions is the *priorityDriver*(entries) function. This documentation will look at how the logic flows from this driver function:

```
//Driver Function for prioritization on a set of UFOEntries
@OntologyEditFunction()
public async  priorityDriver(entries: ObjectSet<Ufoentry>):Promise<void> {
```

To the final prioritization outcome.

Initially, this block of code, in the driver function, isolates the active Priority Configuration. The configureAlgorithm() function is a helper function that returns an array of string arrays based on the active Priority Algorithm instance.

```
let arr  = Objects.search().priorityAlgorithm().all().filter(item => item.mainConfigFlag === 1);
let algo = arr[0];
let config = await this.configureAlgorithm(algo, entries) as string[][];
```

From here, each entry in the entries parameter is iterated over.

```
let ufoArr = entries.all()
for (let i = 0; i < ufoArr.length; i++) {
```

Within this for loop, each UFOEntry has its SBCbump read and/or initialized

```
let bump = ufoArr[i].sbcbump;
if (bump === undefined) {
    bump = 0;
}
```

Still within the for loop, each UFOEntry instance has its priority values read and isolated. Here, “priority values”, refers to the values each UFOEntry instance has in the properties that are being scored. For instance, if HighestMessageUrgency was one of the properties (it often is) the ‘vals’ variable would hold an array containing multiple string arrays, one of them being [HighestMessageUrgency, Critical]. This is done with a helper function getPriorityValsForOneEntry().

```
let vals = this.getPriorityValsForOneEntry(config, ufoArr[i]) as string [][];
```

Still within the for loop, once the values have been pulled, the score needs to be calculated. This is done with a number of helper functions, but is triggered with this line of code in the driver function.

```
let score = this.calculateGlobalScore(vals, config, bump);
```

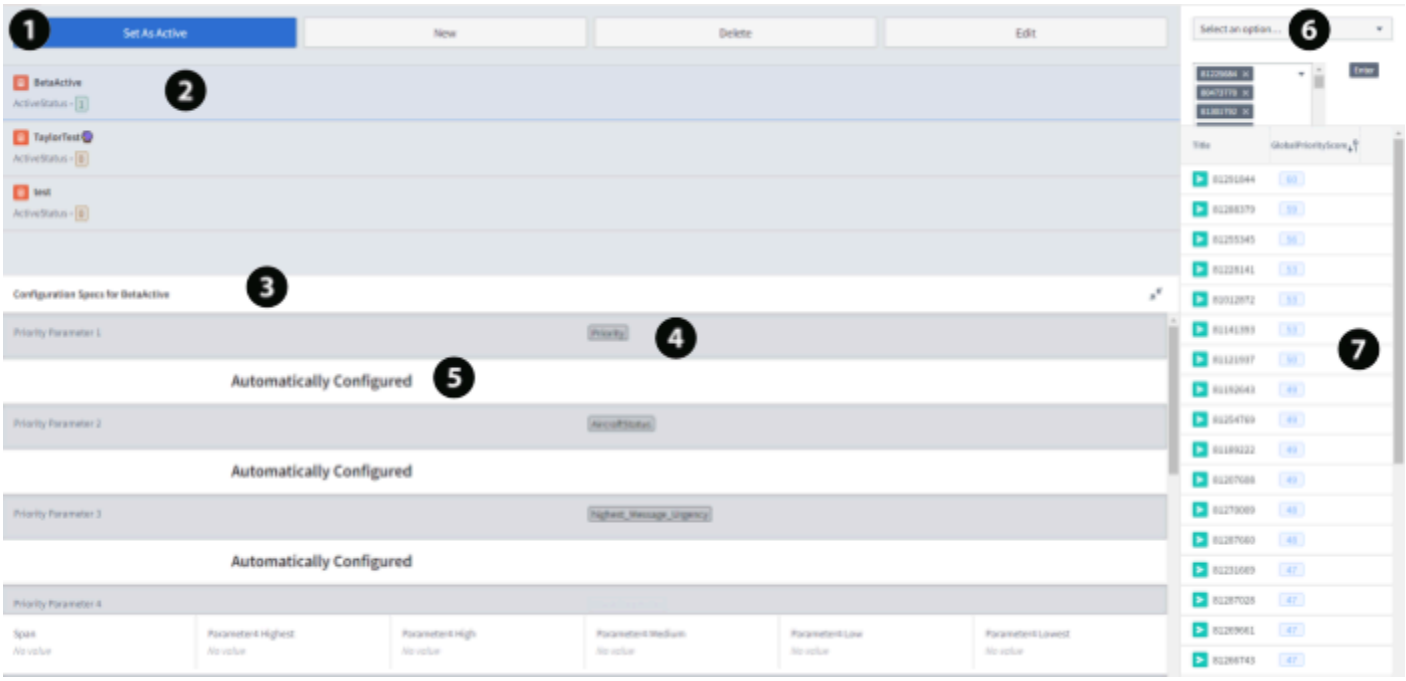
The calculateGlobalScore() helper function determines the weight of each property in the Prioritization Configuration and calls a helper function, calculatePropertyScore() for each property. Based on the nature of the property, different methods will be applied to the property and the respective UFOEntry value. These distinctions are made in calculatePropertyScore(). There are several different categories of properties with respect to prioritization methods. The categories are: Exact Matches, Spans, Dates, BinaryMatches, Keywords and Automatic Configuration. More information about each category can be found in the comments in calculatePropertyScore(). In the calculateGlobalScore() function, the SBC bump is added to each UFOEntry instance’s score.

Still within the for loop, each ufoArr has its globalPriorityScore assigned to the return value from calculateGlobalScore().

```
ufoArr[i].globalPriorityScore = score;
```

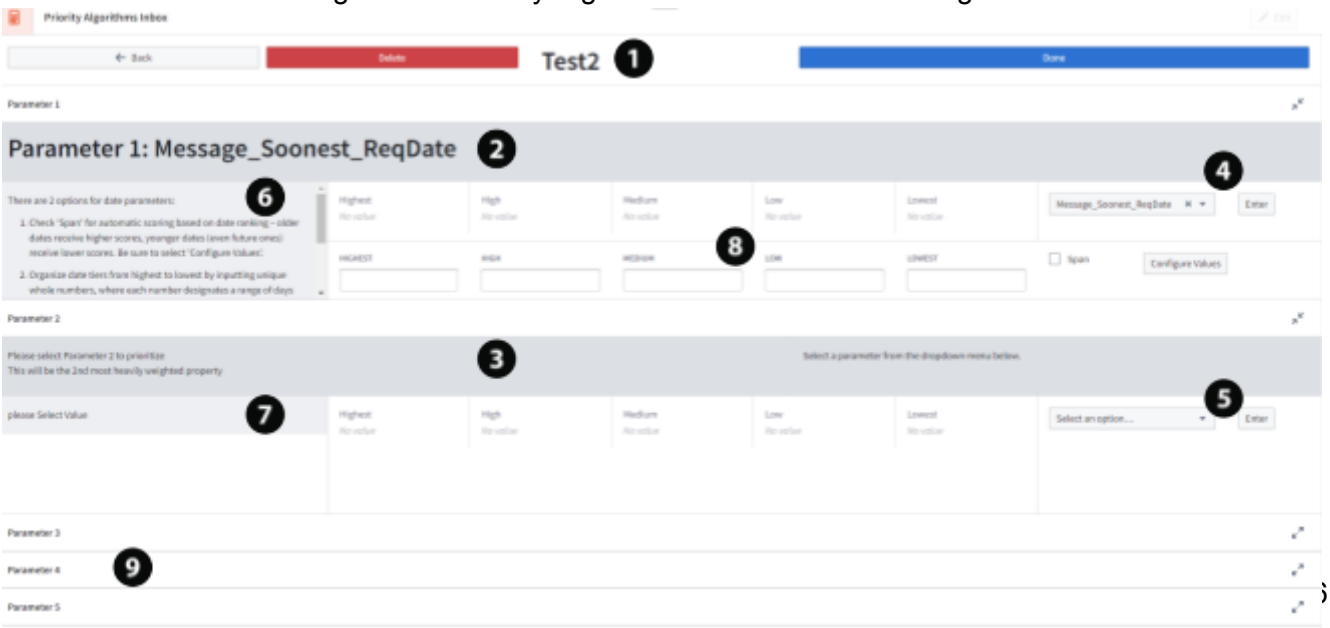
C. Prioritization Dashboard

The Prioritization Workshop Module has two important pages: the Priority Algorithm Management Page and the Priority Algorithm Creation and Edit Page. Below is the Prioritization Algorithm Management Page. Here is where users can view all the Priority Algorithm instances and set them as active, test and view them, and delete them.



1. Button Group
    a. Set as Active
        i. This button triggers the *setMainConfiguration* Action Type. The Active Priority Algorithm instance is set as the MainConfiguration and all other instances are set as nonActive.
    b. New
        i. This button creates a new Priority Algorithm instance and navigates to the Priority Algorithm Creation and Edit Page.
    c. Delete
        i. This button Deletes the selected Priority Algorithm instance.
    d. Edit
        i. This button Navigates to the Priority Algorithm Creation and Edit Page. This page is populated with the selected instance so the instance can be edited.
2. List of Priority Algorithm instances
    a. Here, all the Priority Algorithm instances are listed. Their MainConfig flag is shown where a value of 1 (in green) indicates that the instance is the Active Configuration and 0 (in amber) indicates that the instance is not Active. The selected Priority Algorithm instance in this list is what the Button Group acts on.
3. Configuration Specs
    a. This is a collapsible panel that shows the Configuration of the selected Priority Algorithm instance.
4. Parameter
    a. Here is the first parameter in the Priority Algorithm instance's exact configuration. Subsequent properties are listed below, sequentially.
5. Property Configuration
    a. Here is the exact configuration of the parameter in section 4. In this case, the configuration is handled automatically in the backend.
6. Tested Algorithm
    a. Here is a drop down menu where the user selects from all the existing Priority Algorithm instances for testing.
7. Hypothetical Prioritization
    a. Here is a list, in order of priority, of UFOEntry instances and how they would be scored per the Tested Algorithm.

Below is an image of the Priority Algorithm Creation and Edit Page



1. Priority Algorithm Name
    a. This is the name of the Priority Algorithm instance being edited or created.
2. Selected Parameter 1
    a. This is an example of a parameter that has been selected. In this case, Parameter 1 is Message_Soonest_Reqeust_Date
3. Non-selected Parameter
    a. This is an example of a parameter that has not been Selected.
4. Selected Parameter Dropdown
    a. Here is a dropdown list of UFOEntry properties from which Parameter 1 was selected. It is worth noting that the dropdown is already populated with the selected parameter.
5. Non-selected Parameter Dropdown
    a. This is a dropdown where no selection has been made. To select a property to use as Parameter 2, the user should select an option from the dropdown and hit the Enter button next to the dropdown.
6. Selected Parameter instruction
    a. Here are instructions on how to initialize Parameter 1 L1- L5. These instructions are populated based on the selected property in the dropdown.
7. Non-selected instructions
    a. Since there has been no selection, the instructions are only telling the user to select an option from the dropdown.
8. L1 - L5 Text Entry
    a. Here is where the user actually populates Parameter 1 L1 - L5 based on instructions in section 7. For user ease, L1 is denoted as Highest, L2 as High, L3 as Medium, L4 as Low, L5 as Lowest
9. Other Parameters
    a. Here are all the other Parameters in the Priority Algorithm instance. They are collapsed right now, but they will look very similar to the Selected or Non-Selected Parameter seen above.

Developer Notes

Convert SmallSample Contour to Pipeline Builder or just make a new Transformation in the repository.
Any new Ontology object must start with ‘ufo’ prefix in ID and should be made with Object Storage v2.
Action edits must be updated in Ontology Manager (tag 1.29 → tag 1.30)
Maintenance Actions,(what should be automated

Appendix

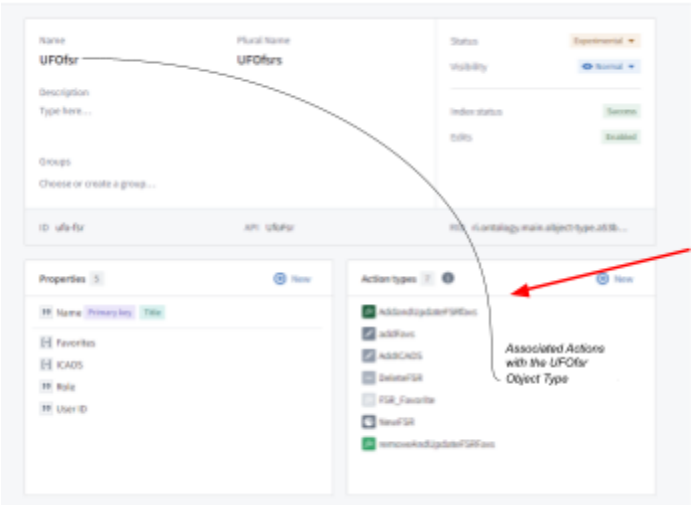


Figure 1

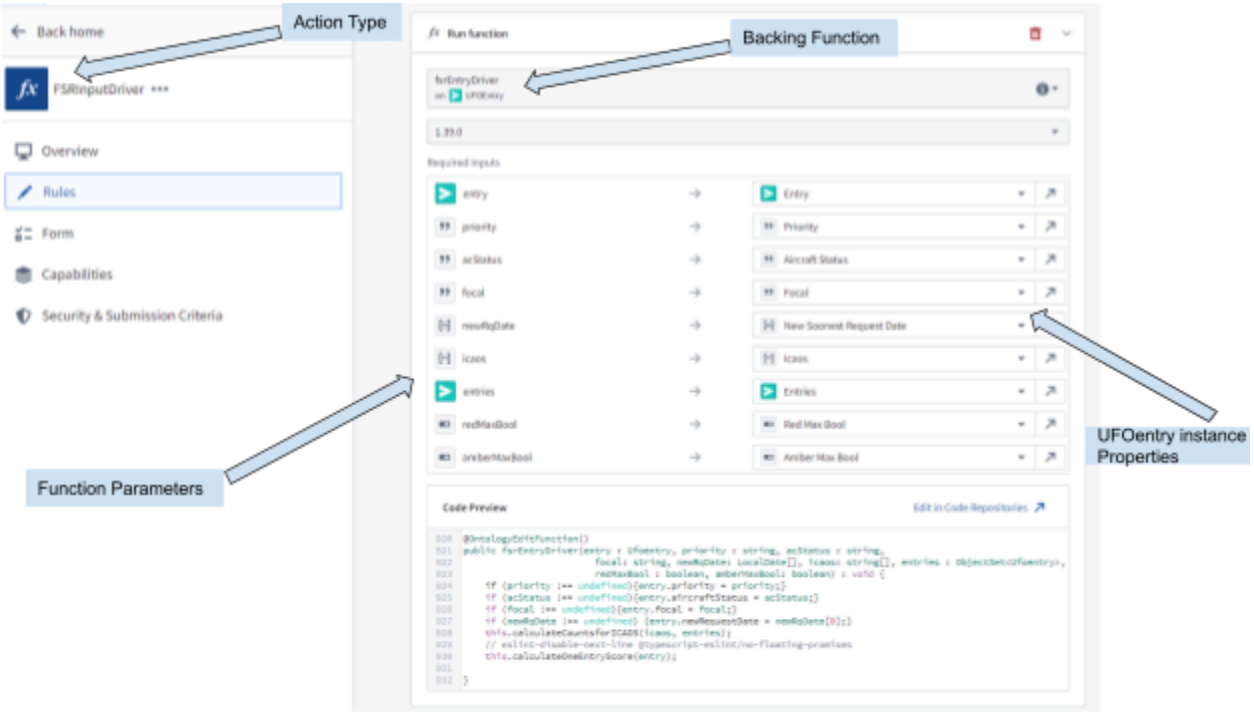


Figure 2

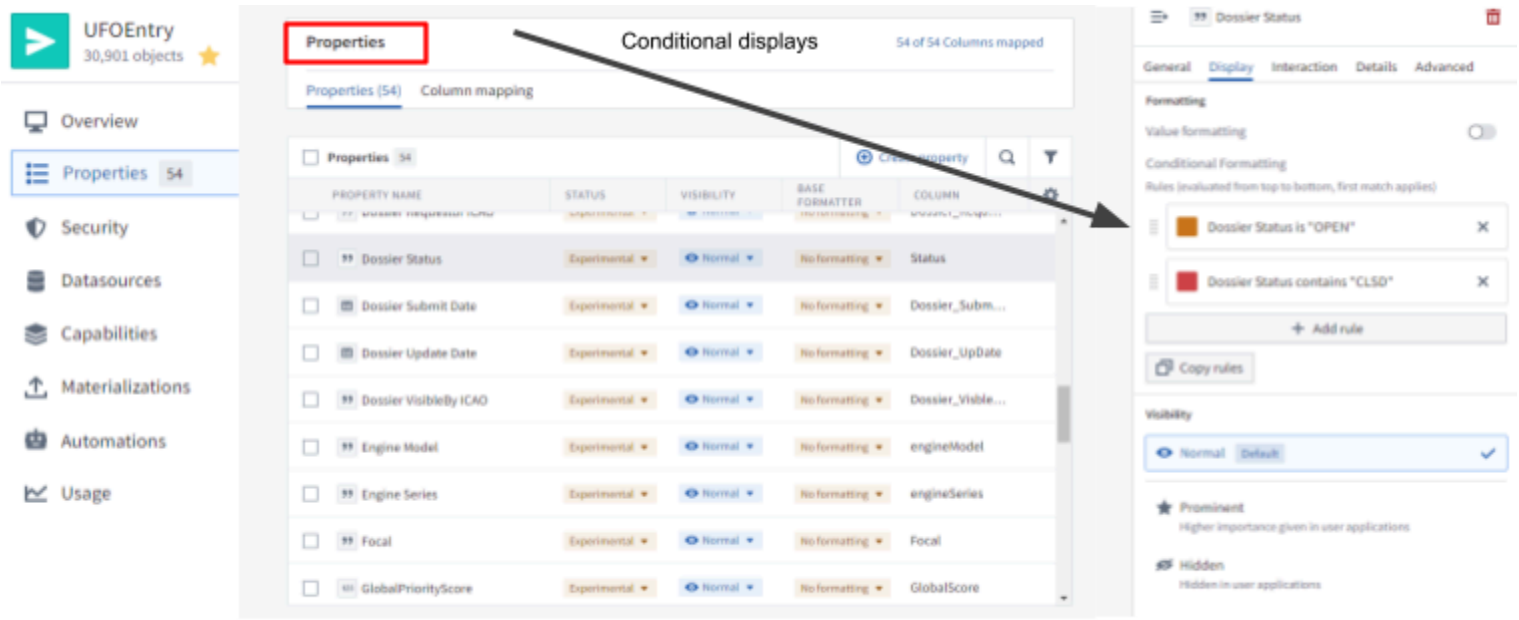


Figure 3

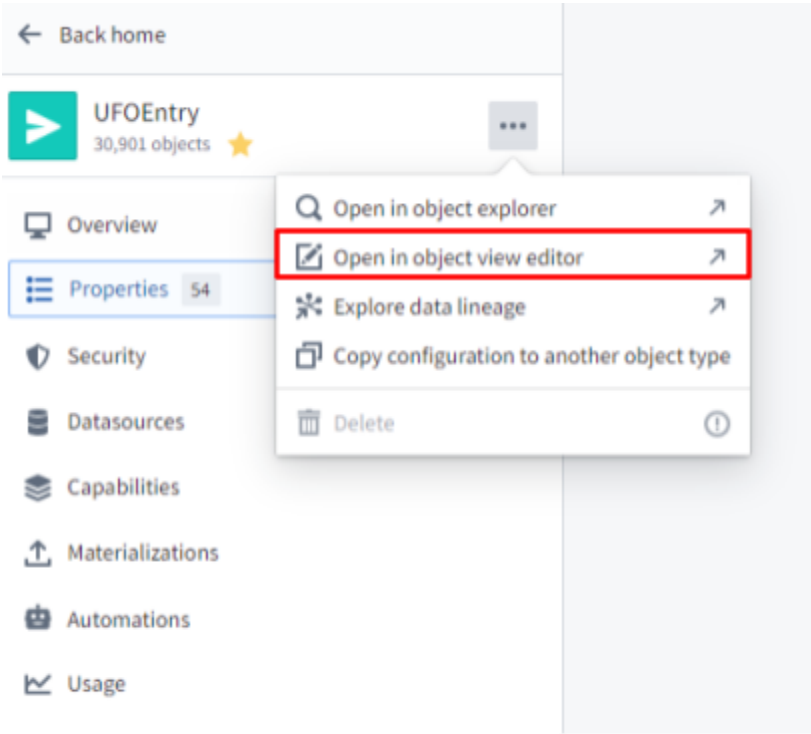


Figure 4